

Printing Receipts, and Getting the Register onto the App Store

For Michael. Written to be followed start to finish, with nothing assumed.

This picks up where *Getting the Register onto Your iPad* (SLICE 9) left off. That document got the app onto one iPad through a cable. This one makes the receipt printer work from inside the app, and gets the app onto the App Store so it can be installed normally on every till.

There is one thing in here with a **1 to 2 week waiting period**, and if you miss it your App Store submission gets rejected. It is Part 5. Please read Part 5 early even if you do everything else later.

Read this part first (3 minutes)

What changed in the app

Before this week, when a cashier tapped **Print**, the iPad jumped out of the register and into Star's PassPRNT app. The receipt printed, and the cashier was left staring at the wrong app with a customer in front of them. There were **seven** places in the register that did this.

They are all gone. The register now talks to the printer itself. The screen never changes.

The fact that changed how I built it

I want to show you this one, because it is a good example of why the "never guess" rule earns its keep.

Your printer is a **TSP143IIIBi**, which is part of Star's **TSP100III** family. Star's programming manual says this, in a note that is easy to miss:

*TSP100III series and TSP100IIU+ do not support `actionPrintText` because these products are **graphics-only printers**. Please use the `actionPrintImage` method.*

In plain English: **your printer cannot be sent text**. It can only be sent pictures. From memory, I would have sent it text — that is how nearly every other receipt printer works. The code would have compiled. It would have passed Apple's review. It would have printed **absolutely nothing**, and you would have found out at the counter.

So the app now draws your receipt on an invisible page, takes a picture of it, and sends the picture. Slightly more work, and the only thing that actually prints on this hardware.

Your two printers — you were right

You said the equipment page already had it. It does. There are two Star printers in there and they do different jobs:

Tag	Model	Job
PRN-COUNTER-01	TSP143IIIBi, Bluetooth, serial 2550923021300119	The front counter printer. Register sales, and it kicks the cash drawer. This slice.
PRN-RECEIPT-01	TSP143IV, Ethernet/CloudPRNT	Auto-prints online orders. Different printer, different job, untouched.

The app is written so it can only ever talk to PRN-COUNTER-01 . There is a test that fails if anyone ever confuses the two.

When the printer breaks

The rule I built to, written at the top of the printing code:

A cash sale is never blocked by a printer.

Out of paper, printer off, Bluetooth dropped, drawer jammed — the sale still completes and is still recorded. You get a plain-English message like *"The printer is out of paper"* rather than a spinning wheel. If the drawer did not open, it tells you so and reminds you that **No Sale** will open it.

What you need in front of you

- Your MacBook Pro, plugged in
- The register iPad (the 12.9-inch Pro, serial DLXQNCCEMGMW3)
- The Star TSP143IIIBi, plugged into power, with paper in it
- Your Apple Developer account login
- About 90 minutes of hands-on time, plus the 1–2 week MFi wait in Part 5

The address question — my recommendation

You asked whether to keep using the Vercel site for testing and switch to GreenwayMarijuana.com at launch, and you asked me to decide.

Yes. Keep Vercel for testing, switch at launch. Here is why, and I checked all three addresses again today rather than trusting last week's notes:

Address	What it does today
<code>https://greenwaymarijuana.com</code>	Loads fine, valid certificate — but it is your old WordPress site . It has no register API.
<code>https://www.greenwaymarijuana.com</code>	Does not load at all. The certificate does not cover <code>www</code> .
<code>https://greenwaywebsite1.vercel.app</code>	The new site . Register API answers correctly.

I owe you a correction here: in the SLICE 9 document I described the apex domain as if it were not the real public site. You corrected me, and you were right — `greenwaymarijuana.com` **is** your live public site today, and the Vercel address is the not-yet-public build of its replacement.

So: build the app pointing at `https://greenwaywebsite1.vercel.app`.

That is the right choice for a specific reason beyond convenience. The register needs a server that has the new API on it. Your apex domain currently serves WordPress, so pointing registers at it today would simply break them. And you want to test before you deploy — which is exactly what the Vercel address is for.

Changing it later is not a big deal. It is one line and a rebuild, about ninety seconds of work plus a new build to the iPads. Nothing is locked in, and you are not committing to anything by using Vercel now.

The switch at launch, when you are ready, is three steps in this order:

1. Point `greenwaymarijuana.com` (and `www`) at the new site instead of WordPress, and let the certificate issue for **both** names.
2. Confirm `https://www.greenwaymarijuana.com` loads without a warning. Right now it does not, and that is worth fixing regardless of the register — customers who type `www` today get a security warning on your business.
3. Rebuild the register app with the new address and push the update.

One caution worth stating plainly: **do not switch the domain and the app on the same day you are busy.** Do it on a slow morning, with the Vercel address still working as a fallback.

Part 1 — Add Star's printer software to the project (Mac, ~15 min)

The Swift code that drives the printer is written and committed. What is not committed is Star's own SDK, because it is their software and it is downloaded by Xcode on demand. You do this once.

Step 1 — Get the latest code

Open **Terminal** on the Mac and run these one at a time:

```
cd ~/greenway/GREENWAY-WEBSITE
git pull
npm ci
```

`npm ci` takes a few minutes and prints a lot. That is normal.

Step 2 — Open the project in Xcode

```
npm run register:build
npx cap sync ios
npx cap open ios
```

Xcode opens. If it starts indexing, let it finish before clicking anything.

*If `npx cap` complains about your Node version, run `node --version`. You need **22 or newer**. Install it from nodejs.org, close Terminal, open a new one, and try again. (My sandbox is stuck on Node 20, which is why I could not run these commands for you.)*

Step 3 — Add the StarXpand SDK

In Xcode:

1. Menu bar → **File** → **Add Package Dependencies...**
2. In the search box at the top right, paste exactly:

```
https://github.com/star-micronics/StarXpand-SDK-ios
```
3. Press Return. A package called **StarXpand-SDK-iOS** appears.
4. Leave **Dependency Rule** at *Up to Next Major Version*.
5. Click **Add Package**. Xcode downloads it — takes a minute or two.
6. A second window asks which target to add it to. Make sure **StarIO10** is checked and the target next to it says **App**. Click **Add Package**.

Step 4 — Check it took

In the left sidebar of Xcode, scroll to the bottom. You should see **Package Dependencies** with **StarIO10** under it, with a version number next to it (for example `2.13.0`). Alongside it you will also see `CapApp-SPM local` and `capacitor-swift-pm 8.5.0` — those two are ours and were already there.

Seeing **StarIO10** in that list means Xcode successfully *downloaded* the SDK.

Step 4b — Check the SDK is actually attached to the app

This is a **different thing** from Step 4, and it is the step people miss.

Step 4 proves Xcode downloaded the SDK. It does **not** prove the SDK is attached to our app. A package can sit in that sidebar list, fully downloaded, and still not be wired into the app — in which case the build fails with:

```
No such module 'StarIO10'
```

To check:

1. In the left sidebar, click the blue **App** icon at the very top.
2. In the main panel, select the **App** target (left column, under TARGETS — not the one under PROJECT).
3. Click the **General** tab.
4. Scroll down to **Frameworks, Libraries, and Embedded Content**.

StarIO10 must be listed there.

- **If it is listed** — you are done, move to Step 5.
- **If the list is empty, or StarIO10 is missing** — click the **+** button underneath the list. A picker opens. Type `StarIO10` in the search box, select **StarIO10**, and click **Add**. It now appears in the list.

Then move to Step 5.

Step 5 — Build it

Press **Cmd-B**. This is the first time this Swift code has ever been compiled — I have no Swift compiler in my sandbox, so I could not do it for you. I wrote it line by line against Star's published manual, but I am telling you plainly that **it has never been**

compiled, so a small error is possible.

- **"Build Succeeded"** → carry on to Part 2.
- **Red errors** → screenshot the errors and send them to me. Do not try to fix them yourself. Errors here will be small and mechanical, and I would rather fix them correctly than have you guess.

Part 2 — Pair the printer to the iPad (~10 min)

This is Bluetooth pairing at the iPad level. It is separate from the app, and you do it once per iPad.

Step 6 — Find the printer's name

1. Turn the printer **off** using the switch on its side.
2. Hold down the **FEED** button on the front, and while holding it, turn the printer **on**.
3. Keep holding until it starts printing, then let go.

It prints a test page. Find the line that says **Dev Name**. It looks like `TSP100-XXXXX`. **Write it down** — you need it in the next step and again in Part 5.

Step 7 — Pair it

On the **iPad**:

1. **Settings** → **Bluetooth**
2. Make sure Bluetooth is **On**
3. Under *Other Devices*, find the `TSP100-XXXXX` name from Step 6
4. Tap it
5. Wait for it to say **Connected**

***If the printer does not appear:** confirm it is on and that the blue light is steady rather than flashing. If it still does not show, hold the small **RST** button on the back while switching the printer on, keep holding through two reset beeps, then release, turn it off and on, and try again. That is Star's factory reset for Bluetooth.*

Step 8 — Lock it down (recommended)

Star recommends turning off **New Pairing Permission** once you are paired, so nobody in the parking lot can pair to your printer. Install **Star Quick Setup Utility** from the App Store, open **Printer Settings** → **Bluetooth Settings**, turn **New Pairing Permission** to **OFF**, and tap **Apply**.

***Read this before you do it.** With that setting off, you cannot pair a new iPad without factory-resetting the printer first. If you plan to add a second till soon, **pair both iPads first**, then turn this off.*

Part 3 — Test printing on the real iPad (~10 min)

Step 9 — Install the new build

With the iPad connected to the Mac by cable, in Xcode choose your iPad from the device menu at the top and press the ► **Play** button. Same as SLICE 9.

Step 10 — Run a test sale

1. Open the register on the iPad
2. Ring up any item and complete a **cash** sale
3. Watch what happens

What should happen: the receipt prints, the drawer pops, and **the screen never leaves the register**. No jump to another app. That is the whole point of this slice.

Step 11 — Test the drawer rules

These are deliberate and worth confirming:

Do this	Drawer should
Cash sale	Open
Refund	Open
Void	Open
No Sale	Open
Reprint a receipt	Stay shut
Print the day report	Stay shut

Reprint not opening the drawer is intentional — otherwise anyone could pop the till by reprinting an old receipt.

Step 12 — Test it failing

Please actually do this one. Open the printer's paper lid so it cannot print, and ring up a cash sale.

Expected: the sale still completes and is recorded, and you get a clear message about the printer. **The sale must not be blocked.** If a printer problem ever stops a sale from completing, that is a bug — tell me immediately.

Part 4 — What is not built yet

✓ **RESOLVED IN SLICE 11 — see** `MICHAEL-slice11-printer-picker.md`

*The printer picker described below has been built and merged (PR #1057). It is on the iPad at **MORE ► ↗ Receipt printer**.*

*Building it also uncovered **two bugs in this slice** that would have stopped pairing from working at all, even once the screen existed: the app rejected Bluetooth printer IDs because it expected a MAC address (Bluetooth uses the printer's name), and the chosen printer was never actually saved. Both are fixed and explained in Parts 4 and 5 of the SLICE 11 doc.*

The rest of this section is left as written, for the record.

Being straight with you about a gap.

There is no **printer settings screen** in the app yet — no screen listing nearby printers for you to pick one. The plumbing is written and tested (`discoverPrinters` , `setPairedPrinterIdentifier`), but nothing calls it.

Until that screen exists, the app has no saved printer, so it falls back to the old PassPRNT route instead of failing a sale. **This means Part 3 may still bounce to PassPRNT**, and that is expected right now, not a failure.

This is one small slice of work. Say the word and it is the next thing I build. I did not bundle it in here because this slice was already large, and I would rather ship the printing engine verified than ship two half-things.

Part 5 — MFi approval ⚠ START THIS EARLY

This is the part with the waiting period. Please read it now even if you do the rest later.

Why this exists

Your printer connects over **Bluetooth Classic**, and Apple requires any accessory like that to be part of its **MFi** program. Your app has to be registered against the printer before Apple will approve it.

Here is the trap: **you do not apply to Apple. Star does, on your behalf.** If you submit your app to the App Store without this, Apple **rejects it**, and you have to wait out the 1–2 weeks anyway. Doing it now costs you nothing.

Straight from Star's own page:

In order to offer your app on the Apple App Store, your app needs to be approved by the Apple MFi program **before you submit it** to the Apple App Store.

Step 13 — Submit Star's form

Go to:

https://star-m.jp/eng/support/s_print/app_regist.html

Fill it in. Here is **exactly** what it asks and what to put:

Developer Information

Field	What to enter
Name	Michael Lyman
E-mail address	The one you check. Star replies here with the PPID.
E-mail (confirm)	Same
Company name	Greenway Marijuana
Area	US & Canada
Address / City / State / Country / Zip	Your Port Orchard, WA business address
Telephone	Include the country code: +1

Application Information

Field	What to enter
Printer(s) your iOS APP communicates with	Tick TSP100IIIBI (under <i>TSP100 Series</i>)
Name of app as it will appear in App Store	Greenway Point of Transaction
APP version	1.0
Bundle Identifier	com.greenwaymarijuana.register
Description of your app	See the paragraph below

⚠ **Tick TSP100IIIBI, not TSP143IV.** *TSP143IV is your online order printer, on Ethernet. The one this app talks to is the Bluetooth TSP100IIIBI.*

Description — the form says single-byte characters only, so plain English, no fancy punctuation. You can paste this:

A point of sale register application for a licensed cannabis retail store in Washington State. Staff ring up in-store sales on an iPad, take payment, and print a customer receipt on a Star TSP143IIIBi over Bluetooth. The app also opens the cash drawer connected to the printer DK port after a cash sale, a refund, a void, or a no sale. The app is for internal store staff use only.

Tick the privacy policy box and click **Confirm**, then submit.

Step 14 — Wait for the PPID

Star files it with Apple, then emails you an **MFi Product Plan ID (PPID)**. It looks like **MFI PPID 123456-7890** .

Expect 1 to 2 weeks. When it arrives, **save it somewhere you will find it again** — you need it in Step 20, and you will need it for every future version of the app.

If two weeks pass with no word, chase them through the contact form at https://star-m.jp/eng/support/s_print/ .

Part 6 — Putting the app on the App Store

Do this once the PPID has arrived.

A decision worth making first

There are two ways to get this app onto your iPads, and the App Store is not obviously the right one:

Route	What it means
App Store (private/unlisted)	Install from the App Store, updates arrive automatically. Requires MFi and Apple review of every version.
Ad Hoc / cable install	What you did in SLICE 9. No review, no MFi, no waiting — but you plug in each iPad for every update, and builds expire annually.

You asked how to get it on the App Store, so that is what is written below. But for two tills in one store, honestly, the cable route is often less friction. The App Store route wins when you stop wanting to touch iPads by hand.

Either way, **do Part 5 now** — it is free and it is the long pole.

Step 15 — Create the app record

On the **Mac**, go to `https://appstoreconnect.apple.com` and sign in.

1. **My Apps** → the **+** → **New App**
2. Platform: **iOS**
3. Name: `Greenway Point of Transaction`
4. Primary language: **English (U.S.)**
5. Bundle ID: pick **com.greenwaymarijuana.register** from the dropdown
6. SKU: `GREENWAY-POT-1` (internal only, never shown)
7. User Access: **Full Access**
8. **Create**

*If the bundle ID is not in the dropdown, it has not been registered yet. Go to `developer.apple.com` → **Certificates, Identifiers & Profiles** → **Identifiers** → **+** → **App IDs** → **App**, enter the description `Greenway Point of Transaction` and the bundle ID `com.greenwaymarijuana.register`, and register it. Then come back.*

Step 16 — Set the version to Distribution

In Xcode: click **App** in the left sidebar → **Signing & Capabilities** → confirm **Automatically manage signing** is ticked and your **Team** is selected.

Step 17 — Archive

1. In the device menu at the top, choose **Any iOS Device (arm64)** — *not* your iPad, and *not* a simulator. Archive is greyed out otherwise.

⚠ **This setting is ONLY for archiving, in this step.** If you leave it on **Any iOS Device (arm64)** and press ► Play, the build fails with "A build only device cannot be used to run this target" — and, because Xcode then has no real device registered, it also fails signing with "Your team has no devices from which to generate a provisioning profile." Both errors are the same cause. Switch the destination back to **your iPad by name** before you press Play again. See *Troubleshooting the build* at the end of this document.
2. Menu bar → **Product** → **Archive**
3. Wait. Five to ten minutes is normal.

The **Organizer** window opens with your archive listed.

Step 18 — Upload

1. In Organizer, select the archive → **Distribute App**
2. Choose **App Store Connect** → **Next**
3. Choose **Upload** → **Next**
4. Accept the defaults on the following screens → **Next**
5. **Upload**

Then wait again. The build shows in App Store Connect as *Processing* for 10 to 60 minutes. You get an email when it is ready.

Step 19 — Fill in the listing

Back in App Store Connect, in your app:

- **Description** — what the app does
- **Keywords**
- **Support URL** — `https://greenwaymarijuana.com`
- **Screenshots** — required, for 12.9-inch iPad Pro. Take them on the iPad with the top button + volume up, and AirDrop them to the Mac.
- **App Privacy** — you must complete this section or you cannot submit
- **Age Rating** — answer honestly; this is a cannabis retail tool
- **Pricing** — Free

Step 20 — ⚠ The PPID. Do not skip this.

This is the step that gets apps rejected.

In your app in App Store Connect, scroll down to **App Review Information**. In the **Notes** box, type your PPID from Step 14, exactly like this:

MFI PPID 123456-7890

(with your real number, obviously)

Star's instruction, word for word:

Submit your App to the Apple App Store through App Store Connect with the MFI PPID number in the Notes section of the App Review Information.

While you are in that box, it is worth adding a note for the reviewer, because this app is useless to them without a login and a printer:

This is an internal point of sale application for a single licensed cannabis retailer in Washington State. It requires a store-issued device credential to operate and connects to a Star TSP143IIIBi Bluetooth receipt printer.
MFI PPID 123456-7890
Demo credentials are provided below.

Then fill in the demo account boxes with a test login. **Reviewers will reject an app they cannot get into.**

Step 21 — Submit

Select your processed build, click **Add for Review**, then **Submit for Review**.

Review usually takes a day or two. Star notes the whole approval can run 1 to 2 weeks including their part.

If something goes wrong

What you see	What it means	What to do
Xcode red errors after Step 5	My uncompiled Swift has a mistake	Screenshot and send to me. Do not guess.
Printer not in Bluetooth list	Not on, or already paired elsewhere	Factory reset: hold RST while switching on, through two beeps
Receipt prints blank	Wrong print mode for a graphics-only printer	Tell me — this is the <code>actionPrintImage</code> issue and it is mine
Prints, but drawer stays shut	Drawer cable, or a reprint (correct)	Check the DK cable; try No Sale
Still jumps to PassPRNT	No printer paired in-app yet	Expected — see Part 4

Apple rejects: MFi/PPID	PPID missing from review notes	Step 20
Sale blocked by printer error	A real bug	Tell me immediately. This must never happen.

What was verified, not assumed

Everything above was checked against a primary source today. Specifically:

- The **graphics-only** limitation is from Star's StarXpand manual, not memory. This is the fact that would have silently broken your receipts.
- The **MFi process, the 1-2 week window, and the PPID going in App Review Notes** are quoted from Star's own MFi page.
- The **registration form fields** were read off the live form. `TSP100IIIBI` is listed there as a real option.
- The **pairing steps** and the **RST reset** are from Star's TSP100IIIBI online manual.
- The **three web addresses** were tested today: the apex serves WordPress with a valid certificate, `www` fails its certificate check, and the Vercel site serves the new app and answers the register API.
- Both printers were read out of the database migration in your own repo.

And what was **not** verified, stated plainly:

- **The Swift plugin has never been compiled.** No Swift toolchain here. First real compile is Step 5 on your Mac.
- **Nothing has been printed on real paper.** No printer here either.
- The printer settings screen does not exist yet (Part 4).

Troubleshooting the build

"A build only device cannot be used to run this target"

+ "Your team has no devices from which to generate a provisioning profile"

These two errors are one problem, and the fix takes five seconds.

They are **not** a code problem, **not** an Apple Developer account problem, and **not** a problem with your signing setup. Nothing is wrong with your account, your team, or the app. Your team shows correctly as *LYMAN'S MARIJUANA L.L.C.* and *Automatically manage signing* is ticked, which is exactly right.

The cause: the destination at the top of the Xcode window is set to **Any iOS Device (arm64)**.

That is not a real device. It is a placeholder that means "*build something generic for later distribution*" — it is what you select to make **Archive** work in Step 17. You cannot **run** on it, because there is no device there to run on. Hence "*a build only device cannot be used to run this target.*"

The signing error is a **knock-on effect of the same thing**, not a second fault. With no real device selected, Xcode has no device to write into a development provisioning profile, so it asks Apple for a profile covering zero devices and Apple declines. That is what "*your team has no devices from which to generate a provisioning profile*" means. It is describing the empty destination, not a broken account.

The fix

1. At the **top centre** of Xcode, click where it says **Any iOS Device (arm64)**.
2. In the dropdown, under a heading like **iOS Device**, choose **your iPad by name**.
3. Press ► **Play**.

Both errors clear together. You do not need to press *Try Again*, sign in again, register a device by hand at developer.apple.com, or change the bundle identifier.

If your iPad is not in that dropdown

Work down this list in order — the first two are the usual answer:

1. **Unlock the iPad** and leave it on the Home screen. A locked iPad often does not appear.
2. **Cable.** Use a real data cable. A charge-only cable is the single most common cause, and it looks identical to a good one.
3. **Trust.** Unplug, plug back in, and on the iPad tap **Trust This Computer**, then enter the passcode.
4. **Developer Mode** must be on: iPad **Settings** ► **Privacy & Security** ► **Developer Mode** ► **On**, then restart the iPad. This only appears after the iPad has been plugged into a Mac running Xcode. (SLICE 9 doc, Step 9.)
5. **Wait 30 seconds** after plugging in. Xcode has to prepare the device the first time, and it is silent while it does.

Why my instructions sent you here — my fault, not yours

Step 17 of this document tells you to select **Any iOS Device (arm64)**, and it is right to, because Archive genuinely requires it. What Step 17 did **not** say is that the setting must be changed **back** to your iPad before pressing Play. If you had been anywhere near Step 17, or had it selected for any other reason, this failure was the guaranteed next thing to happen — and the signing error on top of it makes it look far more serious than it is.

I have added a warning to Step 17 so the next person through does not hit it.

What this error does NOT mean

Worth saying plainly, because the wording is alarming:

- It does **not** mean your Apple Developer account is wrong or unapproved.
- It does **not** mean the £99/\$99 membership has a problem.
- It does **not** mean you need to add your iPad's UDID manually anywhere.
- It does **not** mean the Swift printer plugin failed to compile.

On that last point: **this failure happened before the compiler ran.** So it tells us nothing yet about whether my printer code builds. Once you select your iPad and press Play, watch for what comes next:

- **"Build Succeeded"** then the app launches on the iPad → the Swift plugin compiled. Carry on to Part 2 / Part 3.
- **Red errors naming a `.swift` file** → that is the real first compile of my plugin. Screenshot them and send them to me. Do not fix them yourself.